

zzrenvcheck: Recent Evolution and Pending Work

2026-07-04 15:35 PDT

Abstract

This paper summarises the development of `zzrenvcheck` across releases v0.5.0 through v0.7.0, the design principle that unifies them, and the work that remains. The releases were driven by the migration of research compendia to the `zzcollab` publishing profile, where each change corrected a concrete failure or false positive observed on a real compendium rather than a hypothetical one.

Design principle: declaration, not installation

`zzrenvcheck` complements `renv`; it does not replace it. The two answer different questions.

- `renv` confirms that the lockfile matches what is actually installed in the project library. Its `snapshot()` and `status()` read the installed packages.
- `zzrenvcheck` confirms that the code, `DESCRIPTION`, and `renv.lock` all agree about which packages the project declares.

`zzrenvcheck` is presence and declaration only. It reads source files and the two manifests and never inspects an installed library. That property is load-bearing: it is why the tool needs no installed packages, no container, and no R at all in the shell version, and why it can run on the host or in CI while the container holds the real environment. In a typical pipeline `renv::snapshot` runs where the packages are installed (the container) and the `zzrenvcheck` gate runs where the files live (the host or CI). This framing was made explicit in the README, the package `Description`, and the `check_packages()` help during this cycle.

Recent work

v0.5.0: the fresh argument

`check_packages(fresh = TRUE)` rebuilds `renv.lock` from a clean code scan. Every package used by the code, and its transitive dependencies, is re-resolved to the current repository version and overwritten in the lockfile, and packages no longer used by code are pruned. It routes through `sync_packages()`, which also reconciles `DESCRIPTION`.

This is a deliberate version refresh, distinct from ordinary `auto_fix`, which reconciles presence but preserves existing pins. Because it re-resolves versions from `available.packages()`, it must run where `repos` points at the intended snapshot (in the container, against the pinned Posit Package Manager date), not on a host pointed at a live CRAN mirror. It is intended for use after a base image or repository-snapshot change.

v0.6.0: chunk-aware scanning and `.renvignore`

Two fixes so the code scanner agrees with `renv` and stops miscounting prose.

- Chunk-aware R Markdown and Quarto scanning. `extract_code_packages()` now scans only fenced R code chunks and inline ``r ...`` spans, blanking markdown and LaTeX prose. Previously a namespaced call written in prose, for example `\texttt{Exact::exact.test}` in a methods description or `remotes::install_github(...)` in installation instructions, was miscounted as a code dependency. Real usage in code chunks and inline code is still detected. On the reference compendium this removed two spurious manifest entries (`Exact`, `remotes`) that were pure manuscript prose.
- `.renvignore` support. `find_r_files()` now honours `.renvignore`, so files excluded from `renv`'s dependency scan, for example a host-rendered submission manuscript with its own toolchain, are excluded here too, and the two scanners agree. A gitignore-lite subset is supported (bare filenames, exact relative paths, globs, directory substrings); `renv` continues to apply full gitignore semantics. This was the mechanism that let a compendium exclude a `paper.Rmd` whose `pacman::p_load(...)` would otherwise pull an entire tidyverse closure into the lock.

v0.7.0: role-aware placement and structural dependencies

- Role-aware auto-fix placement. A package used by the compendium's own code in R/ is added to `Imports`; one used only by `analysis/`, `scripts/`, `tests/`, or `vignettes/` is added to `Suggests`. Previously every added package went to `Imports`. The placement is applied in both `handle_auto_fix_description()` and `sync_packages()`.
- Honouring structural dependencies. The new `parse_description_structural()` returns packages declared in `LinkingTo` or `Depends`, for example `Rcpp` for compiled code. These are used via linkage or attachment, not `library()` or `::`, so the code scanner never sees them. They are no longer reported as unused, and `sync/fresh` no longer removes them from `DESCRIPTION` or `renv.lock`. Without this, a `fresh` rebuild would have silently dropped `Rcpp` from a compendium whose compiled core cannot build without it.

All releases preserved the existing test suite and added focused tests; the suite stood at 145 passing checks after v0.7.0.

Pending work

The following items were identified during this cycle and are not yet implemented.

- Add structural dependencies to the lock when missing. v0.7.0 protects `LinkingTo/Depends` packages from removal but does not add them to `renv.lock` when they are absent from both the code scan and the lock. Today a structural dependency must already reach the lock through `DESCRIPTION Imports` (as `Rcpp` does). A more complete `fresh/sync` would resolve and insert the structural closure directly.
- Honour `renv::settings$ignored.packages()`. `zzrenvcheck` currently reads only `.renvignore`, which excludes files, not packages. `renv` also supports a package-level ignore list in `renv/settings.json`. Until `zzrenvcheck` reads it, using that setting reintroduces a scanner disagreement (`renv` ignores the package, the gate still flags it). This is the clean escape hatch for the edge case where a dev tool such as `styler` is genuinely referenced in scanned code but should not be pinned.
- Inline-code and non-R-chunk coverage. Chunk-aware scanning keeps inline ``r ...`` spans but does not scan other engines' chunks; a package used only from, for example, a chunk option evaluation would be missed. This is a narrow gap and has not been observed in practice.
- Version-conflict remediation. The version-synchronisation check added in v0.4.0 reports mismatches between `DESCRIPTION` constraints, `renv.lock`, and code pins but does not auto-resolve them; the resolution policy is still manual.

Relationship to the wider migration

These changes were prerequisites for a clean publishing-profile migration. The chunk-aware scanner and `.renvignore` support removed the false positives that would otherwise bloat a regrown lock; role-aware placement produced a correct `Imports/Suggests` split rather than dumping everything into `Imports`; and honouring `LinkingTo` prevented a compiled dependency from being dropped. The migration playbook records the two-layer manifest model (`renv.lock` pins the closure, `DESCRIPTION` declares roles) that `zzrenvcheck` enforces on the host while `renv` enforces installation in the container.

Rendered on 2026-07-04 at 15:46 PDT. Source: ~/prj/sfw/08-zzrenvcheck/zzrenvcheck/docs/zzrenvcheck-evolution-whitepaper.md